AIにバイブコーディングさせたら「何を実装したか」がわからなくなった話—Unity Visual Scripting を MCP で自動構築する

Bolt、覚えてますか

皆さんは Bolt というライブラリが Asset Store で販売されていたのを覚えているだろうか？

Bolt は Ludiq 社が開発したビジュアルスクリプティングツールで、2017 年に Asset Store でリリースされた。コードを書かずにノードベースのグラフでゲームロジックを組める。Unreal Engine の Blueprint に対する Unity 側の回答として、多くの開発者に支持されていた。有料アセット（$65）でありながら、評価は常にトップクラスだった。

転機は 2020 年。Unity Technologies が Ludiq 社を買収し、Bolt を Unity 公式ツールとして統合することを発表した。同年 7 月、Bolt は Asset Store で**無料化**。そして Unity 2021 LTS から、パッケージ名を `com.unity.visualscripting` に変え、Unity エディタに標準搭載されることになった。

移行の過程では、いくつかの変化があった。

- **名前空間の変更**: `Bolt` → `Unity.VisualScripting`。クラス名も `FlowMachine` → `ScriptMachine`、`StateMachine` はそのまま残った
- **パッケージ管理**: Asset Store の `.unitypackage` から Unity Package Manager（UPM）管理に移行
- **API の整理**: Ludiq 独自のフレームワーク（`Ludiq.Core`, `Ludiq.Graphs` 等）が Unity 側のアーキテクチャに統合された
- **Type Options / Unit Options の自動生成**: Bolt 時代は手動リビルドが必要だったが、Visual Scripting ではプロジェクト設定から自動管理になった

ただ、正直なところ「使ってる人をあまり見ない」というのが自分の肌感覚でもある。C# を書ける開発者にとっては、ノードを繋ぐよりコードを書く方が速い。アーティストやデザイナー向けという位置づけだが、Unity 界隈では Unreal ほどビジュアルスクリプティング文化が根付いていない。

じゃあ何のために Visual Scripting の API を作ったのか。答えは「**AI にノードを組ませるため**」だ。

導入：MCP で Unity を操作する

自分は Unity を外部から操作するための MCP（Model Context Protocol）ブリッジを個人で開発している。Claude Code などの AI エージェントから JSON-RPC でリクエストを投げると、Unity Editor 上でシーン構築・コンポーネント操作・アセット生成などが実行される仕組みだ。まだ完全版は公開していないが、日常的にこれを使って Unity での開発を進めている。

「AI に全部やらせたら、自分のプロジェクトなのに何が起きてるかわからなくなった」

この MCP に Visual Scripting の API を追加しようとした時の話。Claude に「Visual Scripting API でサンプル作って」と投げたら、コードは生成されるしテストも通る。でも Unity Editor で開いた時、自分が「これ何がどうなってるんだっけ」と言い出す始末。

バイブコーディングの代償として、「自分のコードベースなのに実装の全体像が見えない」問題にぶつかった。この記事では、その体験と、そこから得た学びを書く。

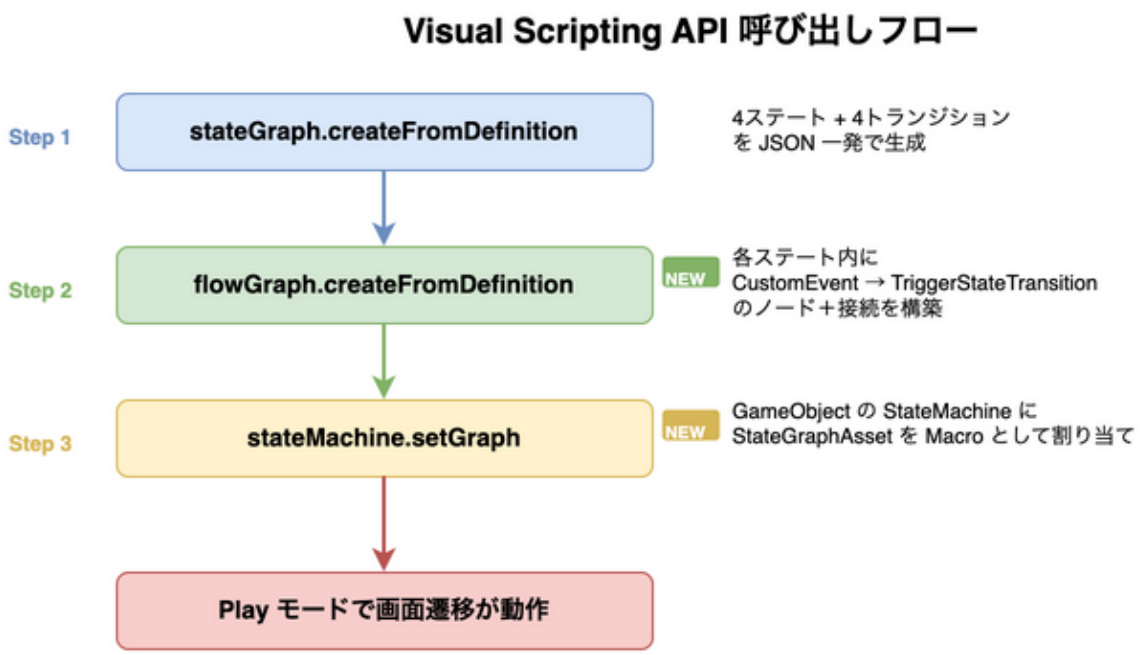

図 1: API 呼び出しフロー

何を作ったのか

MCP の Visual Scripting API。もともと StateGraph の構造（ステートとトランジション）を作る API はあったが、**各ステート内の FlowGraph にノードを追加・接続する API がなかった。**

つまり、AI がステートマシンの骨格を自動生成できても、各画面の内部ロジック（「この画面に入ったらパネルを表示」「ボタンを押したら次の画面へ遷移」）は手動で Unity Editor から組まないといけなかった。

実装した API 一覧

API	役割
`stateGraph.createFromDefinition`	ステート＋トランジションを一括作成（既存）
`flowGraph.addUnit`	FlowGraph にノード（ユニット）を 1 つ追加
`flowGraph.connect`	2 つのポート間を接続
`flowGraph.getUnits`	ユニット・接続の一覧取得
`flowGraph.removeUnit`	ユニットを削除
`flowGraph.createFromDefinition`	JSON 定義からノード＋接続を一括作成
`stateMachine.setGraph`	GameObject の StateMachine に StateGraphAsset を割り当て

デモ：4 画面の UI 遷移を全自動構築する

記事用の小さなデモとして、4 画面（Title → Home → Game → Result）の UI 遷移を Visual Scripting で作った。ゲームでよくある基本的な画面フローだ。

- **Title** — 起動時の画面。「Tap to Start」で Home へ
- **Home** — メインメニュー。「Play」で Game へ
- **Game** — ゲーム本編。「Finish」で Result へ

・ **Result** — 結果画面。「Back to Home」で Home に戻る

ポイントは、**この遷移ロジックを Unity Editor で一切ノードを手動配置せずに、全て MCP API 経由で構築すること**。

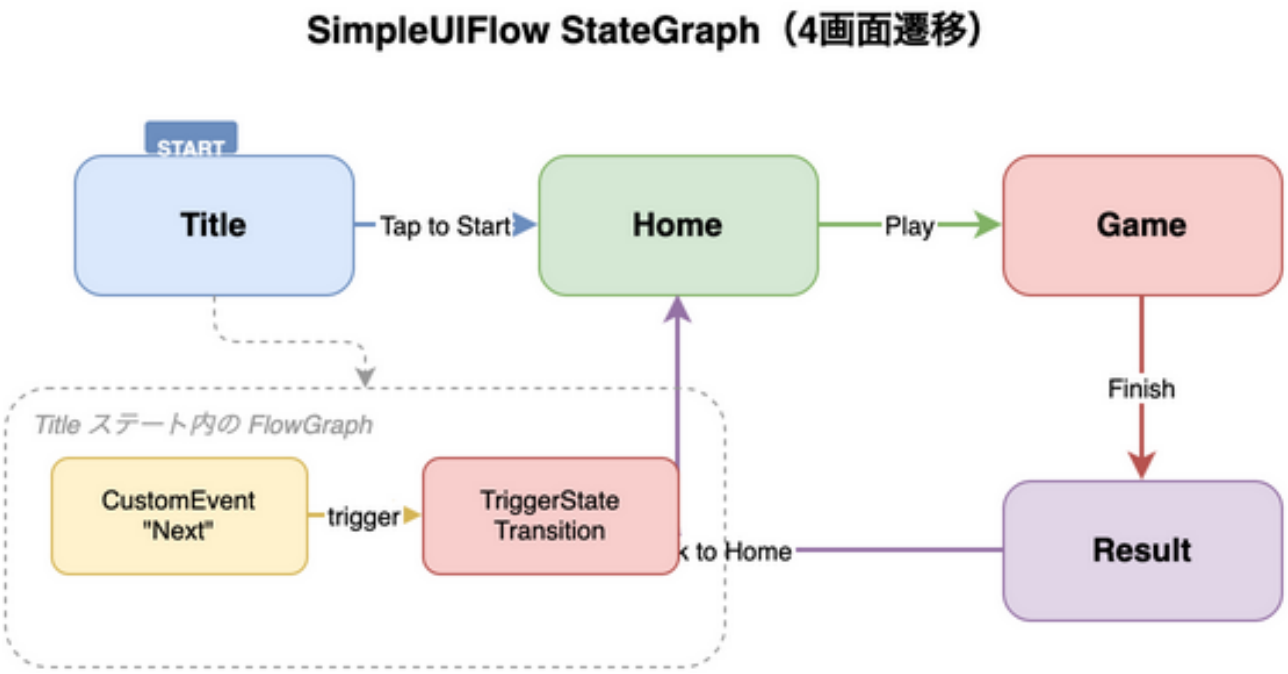

図 2: StateGraph 構造

Step 1: StateGraph を API で一発生成

```
{
  "method": "unity.visualScripting.stateGraph.createFromDefinition",
  "params": {
    "name": "SimpleUIFlow",
    "outputPath": "Assets/VisualScripting/SimpleUIFlow.asset",
    "states": [
      {"name": "Title", "positionX": 0, "positionY": 0, "isStart": true},
      {"name": "Home", "positionX": 300, "positionY": 0},
      {"name": "Game", "positionX": 600, "positionY": 0},
      {"name": "Result", "positionX": 600, "positionY": 250}
    ],
    "transitions": [
      {"source": "Title", "destination": "Home"},
      {"source": "Home", "destination": "Game"},
      {"source": "Game", "destination": "Result"},
      {"source": "Result", "destination": "Home"}
    ]
  }
}
```

これだけで 4 ステート＋4 トランジションの StateGraphAsset が生成される。Unity Editor でダブルクリックすれば、ステートが矢印で繋がったグラフが見える。

TODO: スクリーンショット Unity Editor で SimpleUIFlow.asset を開いた状態。4 つのステートがトランジションで接続されている

ちなみに、最初は現在制作中のゲームの 18 画面遷移（タイトル → ホーム →モード選択 → ソロ/マルチ → 対戦 →リザルト…）を一発で生成してみたが、記事向けには規模が大きすぎたので 4 画面に絞った。18 画面＋ 41 トランジションも JSON 一発で作れることは確認済み。

Step 2: 各ステートに FlowGraph ノードを追加

ここからが今回新しく実装した API の出番。各ステートの内部に「CustomEvent を受信したら次のステートへ遷移する」というフローを組む。

```
{
  "method": "unity.visualScripting.flowGraph.createFromDefinition",
  "params": {
    "assetPath": "Assets/VisualScripting/SimpleUIFlow.asset",
    "stateName": "Title",
    "units": [
      {"unitId": "event", "unitType": "CustomEvent",
       "positionX": 0, "positionY": 0},
      {"unitId": "transition", "unitType": "TriggerStateTransition",
       "positionX": 300, "positionY": 0}
    ],
    "connections": [
      {
        "connectionType": "control",
        "sourceUnitId": "event",
        "sourcePortKey": "trigger",
        "destinationUnitId": "transition",
        "destinationPortKey": "trigger"
      }
    ]
  }
}
```

これを 4 ステート全てに対して実行する。ボタンクリック → C# から `CustomEvent.Trigger()` → Visual Scripting の CustomEvent ノードが受信 → TriggerStateTransition で次のステートへ、という流れだ。

実際にこの API を呼んだ後、`getUnits` で確認すると：

```
[0] Unity.VisualScripting.CustomEvent
    CI:[] CO:['trigger'] VI:['target','name'] VO:[]
[1] Unity.VisualScripting.TriggerStateTransition
    CI:['trigger'] CO:[] VI:[] VO:[]
Connection: [0].trigger --control--> [1].trigger
```

ノードが追加され、ポート間の接続も確認できる。

TODO: スクリーンショット Title ステートを開いた状態。CustomEvent と TriggerStateTransition が trigger ポートで接続されている

Step 3: StateMachine への割り当て

最後に、作成した StateGraphAsset を GameObject の StateMachine コンポーネントに割り当てる。

```
{
  "method": "unity.visualScripting.stateMachine.setGraph",
  "params": {
    "gameObjectPath": "UIFlowManager",
    "assetPath": "Assets/VisualScripting/SimpleUIFlow.asset"
  }
}
```

これも今回新しく実装した API。なかったら Inspector で Source を Macro に変更して、アセットを手動ドラッグしないといけなかった。「API で StateGraph を作れるのに、GameObject に割り当てられないのは片手落ちだろう」と気づいて追加した。

> TODO: スクリーンショット UIFlowManager の Inspector。StateMachine コンポーネントの Source が Macro、SimpleUIFlow.asset が割り当てられている

デモの補助スクリプト

Visual Scripting だけでは UI パネルの表示切替が難しいため、2 つの小さな C# スクリプトを補助として作成した。

- **UIFlowButton** — ボタンの OnClick で `CustomEvent.Trigger()` を発火する。`[RequireComponent(typeof(Bu` で Start 時に自動登録
- **UIFlowManager** — ステート名に応じたパネルの SetActive を切り替える

Visual Scripting 側の仕事は「ステート遷移のタイミング制御」、C# 側の仕事は「UI の表示切替」と「イベントの橋渡し」。この役割分担が、現時点では一番現実的な構成だった。

> TODO: スクリーンショット 完成したシーンの Hierarchy。UICanvas 配下に 4 つのパネル、UIFlow-Manager に StateMachine + UIFlowManager コンポーネント

バイブコーディングで何が起きたか

「動いた！」の裏で起きていたこと

Claude にお願いして実装を進めたら、こういう状態になった。

1. **FlowGraphBuilder.cs** — 500 行の C# が生成された。ユニット追加・接続・削除・一括定義の 5 メソッド。Visual Scripting API のリフレクションを使ってポートを検索・接続する処理
2. **パラメータ定義** — 30 クラス近く追加された
3. **ルーティング** — WebSocket 用と HTTP 用の 2 ファイルに同じルーティングが必要だったが、**片方だけ更新して 30 分ハマった**。API を呼んでも “Method not found” と返ってくる。レジストリには登録されてるのにルーティングに到達しない。原因を切り分けるのに時間がかかった

全部で 6 ファイル、追加コード量は 1000 行超。Claude が書いて、テストして、動いた。

問題は「動いた後」

動いた後に、自分でコードを読み返してみると：

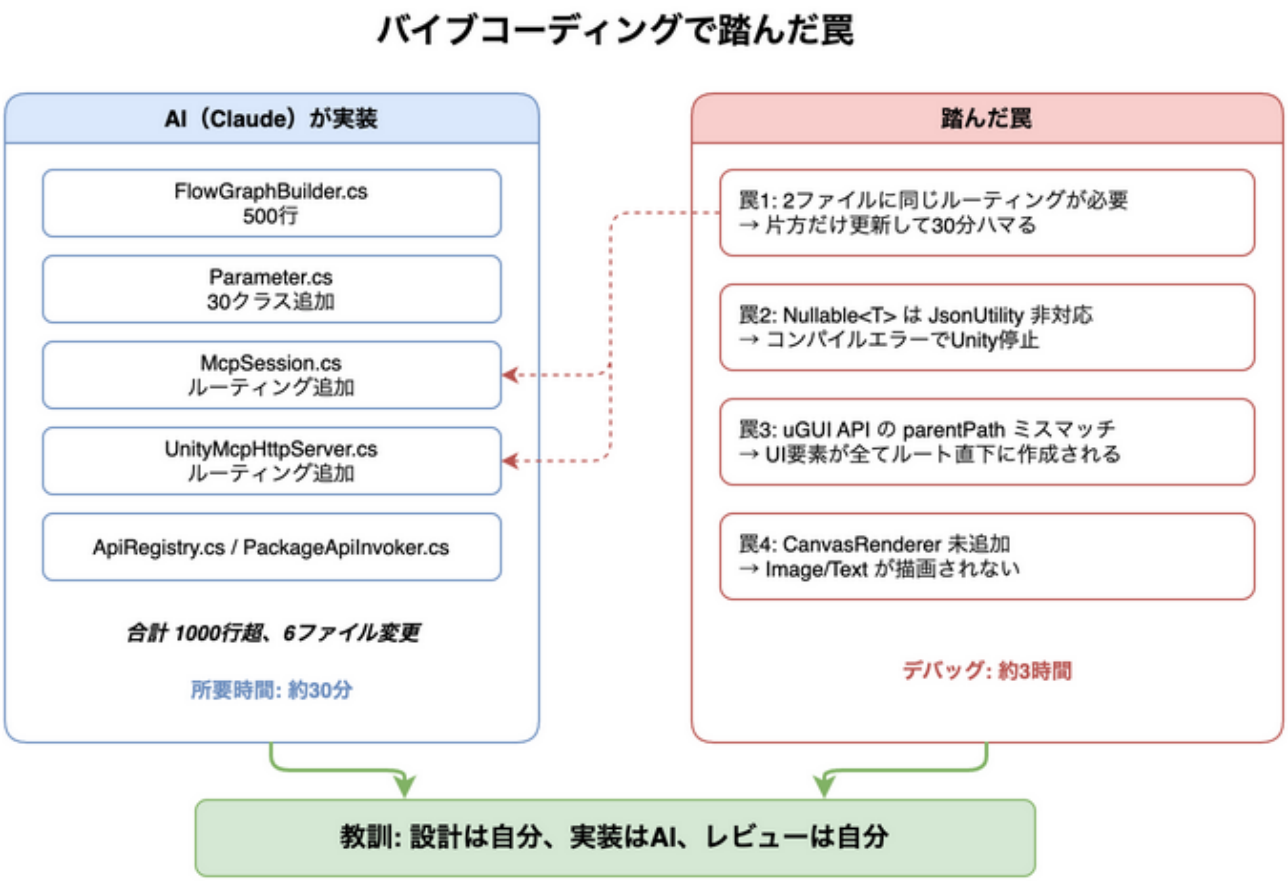

図 3: バイブコーディングの罠

- `CreateUnit()` メソッドが `AppDomain.CurrentDomain.GetAssemblies()` で全アセンブリをスキャンしてる。これ必要？ →短縮名（`OnEnterState`）と完全修飾名の両方に対応するため。なるほど
- `FlowGraphDefaultValue` の `valueType` フィールド。最初 `Nullable<T>` で書いたら `JsonUtility` がシリアライズできなくてコンパイルエラー。Claude が自分で直した
- `EnsureDefined` をリフレクションで呼んでる。Unit のポートが初期化される前に defaultValue を設定しようとして失敗するから。これは自分では思いつかなかった

全部ちゃんと理由がある。でも「理由がある」ことと「自分が理解してる」ことは別の話だった。

Visual Scripting のクセの強さ

正直に言うと、ここまで一筋縄ではいかなかった。Visual Scripting は Bolt 時代からの設計を引きずっていて、外部からプログラマティックに触ろうとするとクセが強い。順番に振り返る。

アセンブリが分離している

Visual Scripting の API は `Unity.VisualScripting.Core`、`Unity.VisualScripting.Flow`、`Unity.VisualScripting.State` の３つのアセンブリに分かれている。外部からこれらを使う場合、asmdef で参照を設定し、さらに `#if UNITY_VISUAL_SCRIPTING` のガードで「パッケージがインストールされている場合のみコンパイル」という条件を付ける必要がある。

パッケージが入っていない環境ではアセンブリ自体が存在しないため、MCP 側のメインアセンブリからは直接参照できない。結果として**リフレクション経由の呼び出し**になる。コンパイル時の型安全性がゼロ。メソッド名を文

字列で指定して `Assembly.GetType()` → `MethodInfo.Invoke()` という泥臭い世界だ。

Unit のポートは「グラフに追加されるまで存在しない」

これが一番ハマったポイント。Visual Scripting の Unit（ノード）は、`new OnEnterState()` でインスタンスを作っただけではポート（controlInputs, controlOutputs, valueInputs, valueOutputs）が**空**。ポートは `Define()` メソッドが呼ばれて初めて生成され、`Define()` は通常、Unit がグラフに追加される時に自動で呼ばれる。

問題は、ノードをグラフに追加する**前**にデフォルト値を設定したいケース。ポートが存在しないのに `defaultValues["value"] = true` を書いても何も起きない。対処として、内部メソッドの `EnsureDefined()` をリフレクションで強制呼び出しする必要があった。

```
var ensureMethod = typeof(Unit).GetMethod("EnsureDefined",
    BindingFlags.NonPublic | BindingFlags.Public
    | BindingFlags.Instance);
if (ensureMethod != null)
    ensureMethod.Invoke(concreteUnit, null);
```

公式ドキュメントにはこの手順の記載がない。Bolt 時代のフォーラムの投稿を漁って、ようやくたどり着いた。

SetMember / InvokeMember はメンバー指定が必須

Visual Scripting で `GameObject.SetActive(bool)` を呼ぶノードは `InvokeMember` という汎用ユニット。しかしこのユニットは、**インスタンス生成時に呼び出し先のメンバー情報（型＋メソッド名）を渡さないとポートが定義されない**。つまり `new InvokeMember()` だけではポートが空で、接続もできない。

JSON で「`SetActive` を呼ぶノードを追加して」と指示しても、メンバー解決のための型情報シリアライズが必要になり、現状の API では対応しきれていない。今回のデモでは `CustomEvent` と `TriggerStateTransition` という、メンバー指定不要なユニットに絞って動作を確認した。InvokeMember 対応は今後の課題。

ステート遷移のトリガー方法が独特

普通のステートマシンなら「条件が真になったら遷移」というイメージだが、Visual Scripting の StateGraph ではトランジション自体にも FlowGraph がある。遷移条件をノードで組むこともできるし、ステート側の FlowGraph から `TriggerStateTransition` を呼んで強制遷移させることもできる。

今回は後者のアプローチを採用した。Visual Scripting だけで完結させようとすると、ボタンの OnClick イベントをノードで拾う必要があり、さらに複雑になる。C# の `CustomEvent.Trigger()` で橋渡しする方が、はるかにシンプルだった。

`JsonUtility` の制約

Unity の `JsonUtility` はシンプルだが制約が多い。`Nullable<T>`（`int?`, `float?`, `bool?`）が使えない。最初 Claude がパラメータを `bool? boolValue` で定義したら、Unity がコンパイルエラーで停止。MCP サーバーが落ちてヘルスチェックも通らなくなり、原因の特定に時間がかかった。

対処として `valueType` フィールド（`"bool"`, `"int"`, `"float"`, `"string"` を文字列指定）を追加し、Nullable を使わない設計に変更した。Unity 固有の制約を知らないと踏む罠で、AI も例外ではなかった。

こうした「Visual Scripting 固有のクセ」は、公式ドキュメントだけでは到底カバーできない領域にある。Bolt 時代のフォーラム、GitHub の Issue、Stack Overflow の断片的な回答を拾い集めて、ようやく動く実装にたどり着いた。AI が書いたコードの中にある `EnsureDefined` のリフレクション呼び出しも、全アセンブリスキャンも、全部この「クセの強さ」への対処だった。

学んだこと

1. AI の実装速度と自分の理解速度は一致しない

Claude は 30 分で 1000 行書ける。でも自分がその 1000 行を理解するには 3 時間かかる。「動いた」を「理解した」と勘違いすると、後でデバッグできなくなる。

2. 構造的な罠は AI も踏む

WebSocket 用と HTTP 用の 2 ファイルに同じルーティングが必要なことに、Claude も最初は気づかなかった。コードベースの構造的な設計判断（なぜ 2 つのサーバー実装があるのか）を理解していないと、片方だけ更新して「動かない」と悩むことになる。人間でも AI でも同じ。

3. バイブコーディングの正しい使い方

「全部任せる」じゃなくて、「設計は自分、実装は AI、レビューは自分」がいいバランスだと思う。今回でいうと：

- **設計**:「FlowGraph 内のノードを追加・接続する API が必要」→これは自分の判断
- **実装**: Unit の型解決、ポート検索、リフレクションでの接続 → Claude の方が速い
- **レビュー**:「なぜ全アセンブリスキャンしてるのか」「Nullable がなぜダメなのか」→ 自分で理解する

この 3 段階をサボると、「自分のプロジェクトなのに何が起きてるかわからない」状態になる。

4. Visual Scripting は「AI が操作する UI」として面白い

人間が手でノードを繋ぐのは面倒だが、AI が JSON で一括生成するなら話は別。18 画面の遷移グラフも一発で作れる。「人間が使いにくいツール」が「AI が操作しやすいツール」に化ける可能性がある。Blueprint 的なビジュアルスクリプティングの新しい活路かもしれない。

まとめ

バイブコーディングは速い。本当に速い。でも「速く動くものができた」と「自分が何を持っているか理解している」はイコールじゃない。

今回、StateGraph 作成 → FlowGraph ノード構築 → StateMachine 割り当てまで、全てを JSON-RPC 経由で自動化できるようになった。4 画面の UI 遷移デモも、API 呼び出しだけで StateGraph の骨格からステート内のノード接続まで一気通貫で構築できた。現在制作中のゲームの 18 画面遷移も一発で生成できることは確認済みだ。

でも一番の収穫は、「自分がレビュワーとして機能しないと、AI の出力は検証されないコードの山になる」という実感だったりする。Visual Scripting のクセの強さは、それを嫌というほど教えてくれた。